CONSTRUINT BMO: MODELS DE LLENGUATGE, RECONEIXEMENT DE VEU I IA LOCAL AMB RASPBERRY PI

“Foto”

Hugo Llorente Vilanova
2n batxillerat A
Institut Torre Roja
Manel López
08/09/2026

ABSTRACT:

The relevance of artificial intelligence lies in its growing presence in everyday life, from voice assistants to automated decision-making systems, yet most users interact with these tools without understanding how they actually work. This paper explores the fundamentals of language models (LLMs), along with speech recognition and speech synthesis mechanisms, in order to demystify the technology behind modern AI assistants. It also analyzes the key differences between cloud-based artificial intelligence and locally-run AI, focusing on privacy, cost, latency, and accessibility, since these two approaches involve fundamentally different trade-offs for the end user. As the central and most extensive part of the project, a functional AI assistant is designed and built using a Raspberry Pi, integrating a local language model (gemma3) with a vision model (moondream), speech recognition software (faster-whisper), wake-word detection (openWakeWord), and speech synthesis (Piper), inspired by the BMO character from Adventure Time. The entire construction process was documented on video, including the technical decisions made and the problems encountered along the way. The performance and limitations of the resulting assistant are then evaluated through direct testing. This project combines theoretical research with a hands-on engineering approach, aiming to understand how AI works beyond its use as an opaque tool.

Keywords: Artificial intelligence, language models, local AI, cloud AI, Raspberry Pi, voice assistant.

RESUM:

La rellevància de la intel·ligència artificial rau en la seva presència creixent en la vida quotidiana, des d'assistents de veu fins a sistemes de decisions automatitzats, tot i que la majoria d'usuaris hi interactuen sense entendre'n realment el funcionament. Aquest treball explora els fonaments dels models de llenguatge (LLM), juntament amb els mecanismes de reconeixement i síntesi de veu, amb l'objectiu de desmitificar la tecnologia que hi ha darrere dels assistents d'IA actuals. També s'analitzen les diferències clau entre la intel·ligència artificial al núvol i la que funciona localment, centrant-se en la privadesa, el cost, la latència i l'accessibilitat, ja que aquests dos enfocaments impliquen compromisos molt diferents per a l'usuari final. Com a part central i més extensa del treball, es dissenya i es construeix un assistent d'IA funcional amb una Raspberry Pi, integrant un model de llenguatge local (gemma3) amb un model de visió (moondream), programari de reconeixement de veu (faster-whisper), detecció de paraula d'activació (openWakeWord) i síntesi de veu (Piper), inspirat en el personatge BMO d'Hora d'Aventures. Tot el procés de construcció s'ha documentat en vídeo, incloent-hi les decisions tècniques preses i els problemes trobats pel camí. Posteriorment, s'avaluen el rendiment i les limitacions de l'assistent resultant mitjançant proves directes. Aquest projecte combina recerca teòrica amb un enfocament pràctic d'enginyeria, amb l'objectiu de comprendre com funciona la IA més enllà del seu ús com a eina opaca.

Paraules clau: Intel·ligència artificial, models de llenguatge, IA local, IA al núvol, Raspberry Pi, assistent de veu.

ÍNDEX

1. INTRODUCCIÓ I OBJECTIUS……1

2. PART TEÒRICA……2

2.1. Què és la intel·ligència artificial……2

2.1.1. Origen i evolució del concepte……2

2.1.2. Tipus d'IA……2

2.2. Models de llenguatge (LLM) i reconeixement de veu

2.2.1. Com funciona un LLM

2.2.2. Reconeixement de veu i síntesi de veu

2.3. IA local vs. IA al núvol

2.3.1. Diferències tècniques

2.3.2. Privadesa, cost i rendiment

2.4. Aplicacions actuals i context

2.5. Debat social/ètic

2.6. Altres conceptes tècnics del sistema

2.6.1. Màquina d'estats finits (Finite State Machine)

2.6.2. Detecció de la paraula d'activació (Wake Word Detection)

3. PART PRÀCTICA

3.1. Disseny i construcció de l'assistent d'IA

3.1.1 Estratègia de desenvolupament

3.1.2 Arquitectura general del sistema

3.1.3 Selecció de maquinari i programari

3.1.4 Aspectes pendents del disseny

3.2. Procés de construcció i posada en marxa

3.2.1. Origen del projecte i primers passos

3.2.2. Revisió i reescriptura del codi original

3.2.3. Instal·lació i configuració dels components de programari

3.2.4. Resolució d'un problema de maquinari: saturació del micròfon

3.2.5. Cerca de la veu de BMO

3.2.6. Trasllat a la Raspberry Pi

3.3. Anàlisi i resultats de l'assistent

3.3.1. Metodologia de les proves

3.3.2. Primeres proves de conversa

3.3.3. Detecció d'un error

4. CONCLUSIONS

5. FONTS D'INFORMACIÓ

5.1. Bibliografia

5.2. Webgrafia

6. ANNEXOS

6.1. Annex 1

6.2. Annex 2

6.3. Annex 3

1. INTRODUCCIÓ I OBJECTIUS

Actualment, vivim un moment de gran transformació tecnològica marcat per l'auge de la intel·ligència artificial. Fa només uns anys, parlar amb una màquina i rebre'n una resposta coherent semblava ciència-ficció; avui, milions de persones fan servir assistents com ChatGPT, Claude o Gemini cada dia, sovint sense plantejar-se com funcionen realment ni què implica que les seves dades viatgin constantment cap a servidors externs.

Aquest interès per entendre què hi ha darrere de la intel·ligència artificial va néixer gràcies a la parella de la meva germana, que em va donar a conèixer un vídeo on un creador de continguts construïa una rèplica funcional de BMO —el personatge d'Hora d'aventures— utilitzant una Raspberry Pi i models d'intel·ligència artificial que funcionaven de manera local. Aquest vídeo em va fer adonar que la IA no és un producte inaccessible ofert per grans empreses, sinó una tecnologia que, amb els coneixements adequats, es pot construir i entendre a petita escala.

A partir d'aquí, com a estudiant de recerca, em vaig proposar anar més enllà de l'ús quotidià d'aquestes eines i investigar-ne el funcionament intern: què és exactament un model de llenguatge, com entén la veu humana, i quines diferències reals hi ha entre una intel·ligència artificial que funciona al núvol i una que funciona de manera local. Aquesta segona qüestió em sembla especialment rellevant ja què la privadesa digital és cada cop més valorada i qüestionada.

Per tal de respondre a aquestes preguntes, aquest treball combina una part teòrica —centrada en el funcionament de la intel·ligència artificial, els models de llenguatge i la comparació entre IA local i IA al núvol— amb una part pràctica què consistirà en construir el meu propi assistent d'intel·ligència artificial local amb una Raspberry Pi, inspirat en el projecte que em va motivar a triar aquest tema.

Objectius del treball:

- Conèixer els fonaments de funcionament dels models de llenguatge (LLM) i dels sistemes de reconeixement i síntesi de veu.
- Dissenyar i construir un prototip funcional d'assistent d'IA amb Raspberry Pi.
- Avaluar el rendiment i les limitacions del prototip mitjançant proves pràctiques.
- Reflexionar sobre la dependència actual de la societat envers la intel·ligència artificial.

2. PART TEÒRICA

2.1. Què és la intel·ligència artificial

2.1.1. Origen i evolució del concepte

El concepte d'intel·ligència artificial tal com el coneixem avui ha nascut formalment l'any 1956, al Dartmouth Summer Research Project, una trobada organitzada per John McCarthy que sol considerar-se el punt de partida oficial d'aquest camp de recerca. Ara bé, les bases teòriques ja hi eren abans: el 1950, Alan Turing havia estat el primer a plantejar la idea d'utilitzar la tecnologia informàtica per simular el pensament i el comportament intel·ligent.

Des d'aleshores, la disciplina ha passat per etapes ben diferenciades. Als anys seixanta i setanta la recerca es va centrar sobretot en sistemes basats en regles i en la lògica simbòlica, un enfocament que va topar amb les seves pròpies limitacions i va donar pas al que es coneix com "l'hivern de la IA": un període en què el finançament es va reduir dràsticament perquè els resultats no estaven a l'altura de les expectatives que s'havien generat. La situació va començar a canviar als anys vuitanta i noranta, amb l'aparició dels sistemes experts i un creixement sostingut de l'aprenentatge automàtic. Ja al segle XXI, el 2012 el deep learning va marcar un abans i un després en el reconeixement d'imatges, i el 2017 va arribar l'arquitectura Transformer, sobre la qual es construeixen avui la majoria dels models de llenguatge.

2.1.2. Tipus d'IA

Avui dia, tota la intel·ligència artificial que existeix realment es classifica com a IA feble o estreta (Narrow AI). Es tracta d'un tipus d'IA que es pot entrenar per fer una tasca molt concreta —sovint més ràpid i millor que una persona—, però que no és capaç de fer res fora d'aquesta tasca per a la qual ha estat dissenyada. Assistents com Siri, Alexa, IBM Watson i, fins i tot, ChatGPT es consideren exemples d'IA feble, ja que tots ells estan limitats a una funció concreta. És el mateix cas dels models utilitzats en aquest treball, gemma3 i moondream, cadascun especialitzat en una única tasca (text i visió, respectivament) (IBM, 2025).

En canvi, la IA forta o general (AGI) faria referència a un sistema capaç de dur a terme qualsevol tasca intel·lectual que pugui fer una persona, amb la mateixa flexibilitat. Aquest tipus d'IA, però, avui en dia és purament teòrica: no existeix, de moment, cap sistema d'intel·ligència artificial general real (IBM, 2025).

2.2. Models de llenguatge (LLM) i reconeixement de veu

2.2.1. Com funciona un LLM

Un model de llenguatge (Large Language Model o LLM) és un programa entrenat amb quantitats enormes de text, la funció del qual és predir quina paraula és més probable que vingui després en una conversa. No "sap" coses com ho faria una base de dades amb informació guardada; en realitat, genera el text a poc a poc, paraula per paraula (o, més exactament, en fragments anomenats tokens), a partir dels patrons que ha après durant l'entrenament.

Un punt important és la diferència entre un model **local** i un model **al núvol**. Un model com ChatGPT es troba en servidors externs: cada missatge que li enviem viatja per internet fins allà, i la resposta torna pel mateix camí. El model local, en canvi, és una versió molt més petita que funciona directament dins del nostre propi ordinador, sense enviar cap dada a fora. A canvi de ser menys potent que els grans models del núvol, té l'avantatge de ser gratuït, privat i de funcionar encara que no hi hagi connexió a internet.

En aquest projecte, el programa que s'encarrega de carregar i fer funcionar aquests models de manera local és **Ollama**. El model concret que s'utilitza és **gemma3: 1b**: el "1b" indica que té mil milions de paràmetres, és a dir, els valors interns que s'ajusten durant l'entrenament. És una xifra petita si la comparem amb els grans models del núvol, que poden arribar a tenir centenars de milers de milions de paràmetres.

2.2.2. Reconeixement de veu i síntesi de veu

Fonaments del so digital

Abans de veure com un ordinador "entén" la veu, cal explicar tres conceptes bàsics del so digital.

El **mostreig** (sample rate) és la manera com un micròfon capta el so. En realitat, no grava un so continu, sinó que el mesura moltes vegades cada segon i guarda cada mesura com un número. Per exemple, quan diem que un so s'enregistra a 44.100 Hz, vol dir que el micròfon ha pres 44.100 mesures en un sol segon.

El **guany** (gain) és, senzillament, l'amplificació que rep el so quan es grava. Funciona una mica com el volum d'un altaveu, però al revés: en comptes d'apujar el so quan surt, s'apuja abans d'entrar a l'ordinador.

Finalment, la **saturació** (clipping) apareix quan aquest guany és massa alt. El so "s'escapa" del límit màxim que es pot guardar i, en lloc del so real, el que sentim és un soroll o una mena de crepitació. És una mica el que passa amb una fotografia sobreexposada: si hi ha massa llum, la imatge es "crema" i es perd el detall.

Reconeixement de veu (Speech-to-Text)

El reconeixement de veu és el procés que converteix la veu d'una persona en text escrit. L'ordinador no "sent paraules" tal com ho faria una persona: primer transforma l'àudio en una imatge anomenada espectrograma, que mostra com sona aquell so al llarg del temps i en diferents freqüències. Després, una xarxa neuronal —entrenada prèviament amb moltíssimes hores de veu humana ja transcrita— va endevinant, tros a tros, quines paraules corresponen a aquell patró de so.

En aquest projecte s'ha fet servir un programa anomenat **faster-whisper**, basat en el model Whisper. En concret, s'utilitza la versió "base", que és més petita i ràpida que altres versions més precises però lentes. A més, es fa servir en mode int8, que vol dir que els càlculs interns són més simples per guanyar velocitat, encara que això comporti una petita pèrdua de precisió. Té sentit fer-ho així perquè el que interessa és que l'assistent respongui ràpidament, no que transcrigui de manera perfecta.

Models de visió

Els models de visió (Vision-Language Models) són una variant dels models de llenguatge que, a més de text, també poden rebre una imatge com a part de la pregunta. La imatge es transforma igualment en una mena de tokens, de manera que la mateixa xarxa neuronal que genera text també pot "raonar" sobre el que hi ha en la imatge. En aquest projecte s'utilitza el model **moondream**, que entra en funcionament quan l'assistent fa una fotografia.

Síntesi de veu (Text-to-Speech)

La síntesi de veu fa el procés contrari al reconeixement de veu: agafa un text escrit i el converteix en àudio parlat. Primer, el text es descompon en fonemes, que són els sons més bàsics de la parla. Després, un model entrenat amb gravacions d'una veu humana genera l'àudio corresponent, intentant que soni el més natural possible. En aquest projecte s'utilitza **Piper**, on la "veu" que sentim (guardada en un arxiu .onnx) és, literalment, un model entrenat a partir de gravacions d'una persona real.

2.3. IA local vs. IA al núvol

2.3.1. Diferències tècniques

La diferència de fons entre aquests dos models rau en on es processa la informació. En la IA al núvol, les dades viatgen fins a un servidor extern, s'hi processen i el resultat torna a l'usuari; en la IA local (o edge AI), en canvi, tot el processament té lloc al mateix dispositiu, sense que calgui enviar res a tercers.

2.3.2. Privadesa, cost i rendiment

Aquesta diferència, aparentment tècnica, té conseqüències molt pràctiques. Des del punt de vista de la privadesa, enviar dades a serveis de tercers al núvol comporta un risc que, segons quina informació estigui en joc, pot arribar a ser inacceptable. Pel que fa al rendiment, la IA al núvol compta amb molta més capacitat de càlcul, cosa que fa possibles aplicacions ràpides i a gran escala; els dispositius locals, en canvi, tenen menys potència, i això obliga a treballar amb models més petits i limitats. I quant a l'accessibilitat, mentre els serveis al núvol estan a l'abast de qualsevol persona amb connexió a internet, la IA local exigeix disposar del maquinari adequat.

2.4. Aplicacions actuals i context

Avui dia, els assistents de veu formen part del dia a dia de moltes persones. L'any 2026, els ecosistemes basats en veu ja s'han fet un lloc a les llars, als llocs de treball, als automòbils i fins i tot en l'àmbit de la salut, amb usos que van des de controlar llums i electrodomèstics fins a programar reunions o vigilar la seguretat de casa. Entre les opcions comercials amb més implantació hi ha Alexa, que domina el control de la llar intel·ligent gràcies a la compatibilitat amb més de 100.000 dispositius, i Google Gemini, que lidera el control per veu dins l'ecosistema de Google.

Tot i el pes d'aquestes plataformes comercials, els darrers anys també ha crescut l'interès per alternatives locals i de codi obert. Un exemple és Home Assistant, un projecte que fa temps que treballa per oferir un assistent de veu local, obert i respectuós amb la privadesa, i que avui permet connectar-se a pràcticament qualsevol model de llenguatge (LLM), tant al núvol com executat localment amb eines com Ollama. Iniciatives com aquesta demostren que construir assistents d'IA locals —com el que es desenvolupa en aquest treball— no és només un exercici acadèmic, sinó una tendència real dins la comunitat tecnològica.

2.5. Debat social i ètic

L'ús massiu de la intel·ligència artificial en el dia a dia no està exempt de controvèrsia, i un dels debats que hi genera més soroll és el de la privadesa de les dades. A mesura que el programari d'IA es fa més sofisticat i s'infiltra més en la nostra vida quotidiana, el ventall de dades personals que aquests sistemes poden arribar a recollir s'eixampla ràpidament: dades biomètriques, historial de navegació, informació financera, preferències de compra... sovint sense que l'usuari sigui del tot conscient de la quantitat d'informació que s'està recopilant.

Aquesta preocupació es fa especialment evident en el cas dels assistents de veu. Un estudi qualitatiu sobre l'ús d'assistents d'IA va detectar entre els usuaris una inquietud considerable per la possible fuita de dades delicades, motiu pel qual molts professionals prefereixen recórrer a assistents autoallotjats (self-hosted) —és a dir, sense la intervenció de cap tercer— com a alternativa més privada. Aquesta troballa enllaça directament amb l'objectiu d'aquest treball: esbrinar si un assistent d'IA local pot oferir una alternativa real i funcional als assistents comercials del núvol, sense haver de sacrificar la privadesa pel camí.

Amb tot, alguns estudis apunten que, malgrat aquesta preocupació generalitzada, sovint no es tradueix en un canvi real de comportament: és el que es coneix com la "paradoxa de la privadesa", que s'explica en bona part per l'atractiu de la comoditat i per interfícies que fan difícil localitzar i entendre els controls de privadesa. Aquest fet obre una reflexió interessant per al treball: fins a quin punt som conscients, com a societat, del cost real —en termes de privadesa— que suposa la comoditat que ens ofereixen els assistents d'IA que fem servir cada dia.

2.6. Altres conceptes tècnics del sistema

2.6.1. Màquina d'estats finits (Finite State Machine)

Una màquina d'estats és un sistema que, en cada moment, només pot trobar-se en una situació concreta d'entre una llista tancada de possibilitats, i que va canviant d'una a una altra segons el que va passant. Un bon exemple és un semàfor: només pot estar en vermell, ambre o verd, mai "una mica dels dos" alhora.

En aquest projecte, l'assistent funciona de la mateixa manera: en tot moment només pot trobar-se en un d'aquests estats: **IDLE** (en espera), **LISTENING** (escoltant), **THINKING** (pensant), **SPEAKING** (parlant), **CAPTURING** (fent una fotografia) o **ERROR**. Gràcies a aquest disseny, la interfície sap sempre exactament què ha de mostrar en cada moment, sense necessitat de deduir-ho ni d'improvisar.

2.6.2. Detecció de la paraula d'activació (Wake Word Detection)

Es tracta d'un programa petit i eficient que està escoltant el micròfon en tot moment, però que no entén el llenguatge en general: només està entrenat per reconèixer un so molt concret. El seu funcionament consisteix a comparar fragments curts de so —de menys d'un segon— amb el patró que ha après durant l'entrenament, i a partir d'aquesta comparació genera una puntuació de similitud. Quan aquesta puntuació supera un cert llindar, s'activa la resta del sistema.

Aquest mètode és útil perquè permet que l'assistent estigui sempre "escoltant" sense gastar gaires recursos, ja que reconèixer un únic so és molt més senzill —i, per tant, molt més lleuger— que entendre tot el que es diu. En aquest projecte s'utilitza **openWakeWord**, que analitza fragments d'àudio de 80 mil·lisegons i activa el sistema quan la puntuació de similitud en supera 0,5.

3. PART PRÀCTICA

3.1. Disseny de l'assistent d'IA

3.1.1. Estratègia de desenvolupament

El disseny final de l'assistent preveu integrar-lo dins una carcassa física inspirada en el personatge BMO, executant-se sobre una Raspberry Pi 5. Tanmateix, vaig decidir no desenvolupar i depurar el sistema directament sobre aquest maquinari, ja que això comporta inconvenients importants: els temps de compilació i càrrega de models hi són més llargs, la memòria RAM i la capacitat de càlcul disponibles són inferiors a les d'un ordinador d'escriptori, i sovint resulta incòmode treballar-hi sense un teclat i una pantalla connectats de manera permanent. A més, si un error apareix mentre es treballa directament sobre la Raspberry Pi, es fa difícil saber si la causa és un problema del meu codi o bé una limitació pròpia del maquinari ARM, cosa que alenteix molt la depuració.

Per aquest motiu, vaig optar per una estratègia de desenvolupament en dues fases clarament diferenciades. En primer lloc, vaig decidir deixar tot el flux de funcionament de l'assistent —és a dir, la cadena completa que va des de la detecció de la paraula d'activació, passant per l'enregistrament de la veu, la seva transcripció, la generació d'una resposta pel model de llenguatge, fins a la síntesi final en veu— funcionant de manera fiable en un ordinador portàtil amb el sistema operatiu Ubuntu 24.04 LTS (arquitectura x86_64), utilitzat exclusivament com a banc de proves de desenvolupament. Només un cop assolit aquest primer objectiu vaig preveure migrar el sistema a la Raspberry Pi 5 (arquitectura ARM64), moment en què, en teoria, només hauria de quedar per resoldre allò que és específic d'aquell maquinari concret, com ara la integració de la càmera o l'adaptació als recursos més limitats de la placa.

Aquesta manera de procedir és habitual en projectes d'enginyeria de programari que combinen software i hardware específic: separar, tant com sigui possible, els problemes de lògica del programa dels problemes derivats del dispositiu final, per poder-los abordar per separat i amb més facilitat de diagnòstic.

3.1.2. Arquitectura general del sistema

Per organitzar el comportament de l'assistent vaig adoptar el disseny d'una màquina d'estats finits (vegeu apartat 2.6.1), en què el sistema només pot trobar-se, en cada instant, en un dels estats següents: IDLE (en espera), LISTENING (escoltant), THINKING (pensant), SPEAKING (parlant), CAPTURING (fent una fotografia) o ERROR. Cada un d'aquests estats es correspon amb una expressió facial diferent de la pantalla de l'assistent, de manera que la interfície sap sempre, sense necessitat de deduir-ho, quina cara ha de mostrar en cada moment.

Ara bé, un assistent de veu no pot limitar-se a executar aquestes tasques una darrere l'altra de manera estrictament seqüencial, ja que hi ha accions que han de succeir en paral·lel. Per exemple, mentre l'assistent està parlant, la pantalla ha de continuar animant la boca en sincronia amb l'àudio, i alhora el sistema ha de poder detectar si l'usuari vol interrompre'l. Per resoldre això, vaig fer servir diversos fils d'execució (threads) simultanis: un fil s'encarrega de dibuixar la cara i gestionar la finestra gràfica, un altre escolta contínuament el micròfon i gestiona tota la lògica de la conversa, un altre reprodueix les frases de la resposta a mesura que estan a punt (per tal que l'assistent pugui començar a parlar abans que el model de llenguatge hagi acabat de generar tota la frase), i encara un altre s'encarrega de reproduir sons ambientals mentre el sistema espera la resposta del model. Com que tots aquests fils comparteixen la mateixa memòria del programa, vaig haver de tenir cura que no interferissin entre ells, per exemple evitant que un fil modifiqui una dada que un altre fil utilitza en aquell mateix instant.

3.1.3. Selecció de maquinari i programari

Pel que fa al maquinari, vaig triar els components que es recullen a la Taula 3.1, pensats per formar un dispositiu compacte, portàtil i amb prou autonomia per a funcionar de manera independent un cop muntat dins la carcassa de BMO. Destaco, en particular, la inclusió d'un SAI (sistema d'alimentació ininterrompuda), pensat per evitar que un tall elèctric sobtat pugui corrompre les dades de la targeta d'emmagatzematge de la Raspberry Pi mentre el sistema està en marxa, i d'una pantalla tàctil, on es mostra la carota animada que reacciona segons l'estat de la màquina d'estats descrita a l'apartat anterior.

Taula 3.1. Components de maquinari de l'assistent

Component	Funció
Raspberry Pi 5	Miniordinador que fa de cervell del sistema, executant el codi i els models d'IA
Raspberry Pi Camera Module v2	Càmera integrada per a la captura d'imatges, utilitzada pel model de visió
Pimoroni NVMe Base Duo	Placa d'expansió que permet connectar fins a dues unitats d'emmagatzematge SSD NVMe
Geekworm X1203 5V UPS Shield	Sistema d'alimentació ininterrompuda (SAI): proporciona energia de reserva mitjançant bateries per evitar apagades sobtades en cas de tall elèctric, protegint el sistema operatiu de possibles corrupcions de dades
Freenove 5 Inch Touchscreen Monitor	Pantalla tàctil on es mostra la carota animada de BMO i els seus diferents estats
Mini USB Microphone	Micròfon per a la captura de veu de l'usuari
Bateria de liti 3.7V	Font d'alimentació portàtil del dispositiu
Imants rodons 5x2mm (x16)	Fixació de les peces impreses de l'estructura

Pel que fa al programari, vaig triar els components que es recullen a la Taula 3.2. En la majoria dels casos, la tria ja venia condicionada pel projecte original que em va servir de punt de partida (vegeu apartat 3.2.2), però en cada cas vaig valorar si mantenir-la o substituir-la:

Taula 3.2. Components de programari de l'assistent

Component	Funció
Ollama	Motor que gestiona la càrrega i l'execució local dels models de llenguatge, sense necessitat de connexió a internet
gemma3:1b	Model de llenguatge (LLM) encarregat de generar les respostes de text de l'assistent
moondream	Model de visió (Vision-Language Model), s'activa quan l'assistent fa una fotografia i ha de descriure o respondre preguntes sobre la imatge
faster-whisper	Programari de reconeixement de veu (Speech-to-Text), basat en el model Whisper, que transcriu la veu de l'usuari a text
openWakeWord	Sistema de detecció de la paraula d'activació, que manté el micròfon escoltant contínuament fins detectar el so que inicia la conversa
Piper	Programari de síntesi de veu (Text-to-Speech), que converteix la resposta generada en text a un àudio parlat

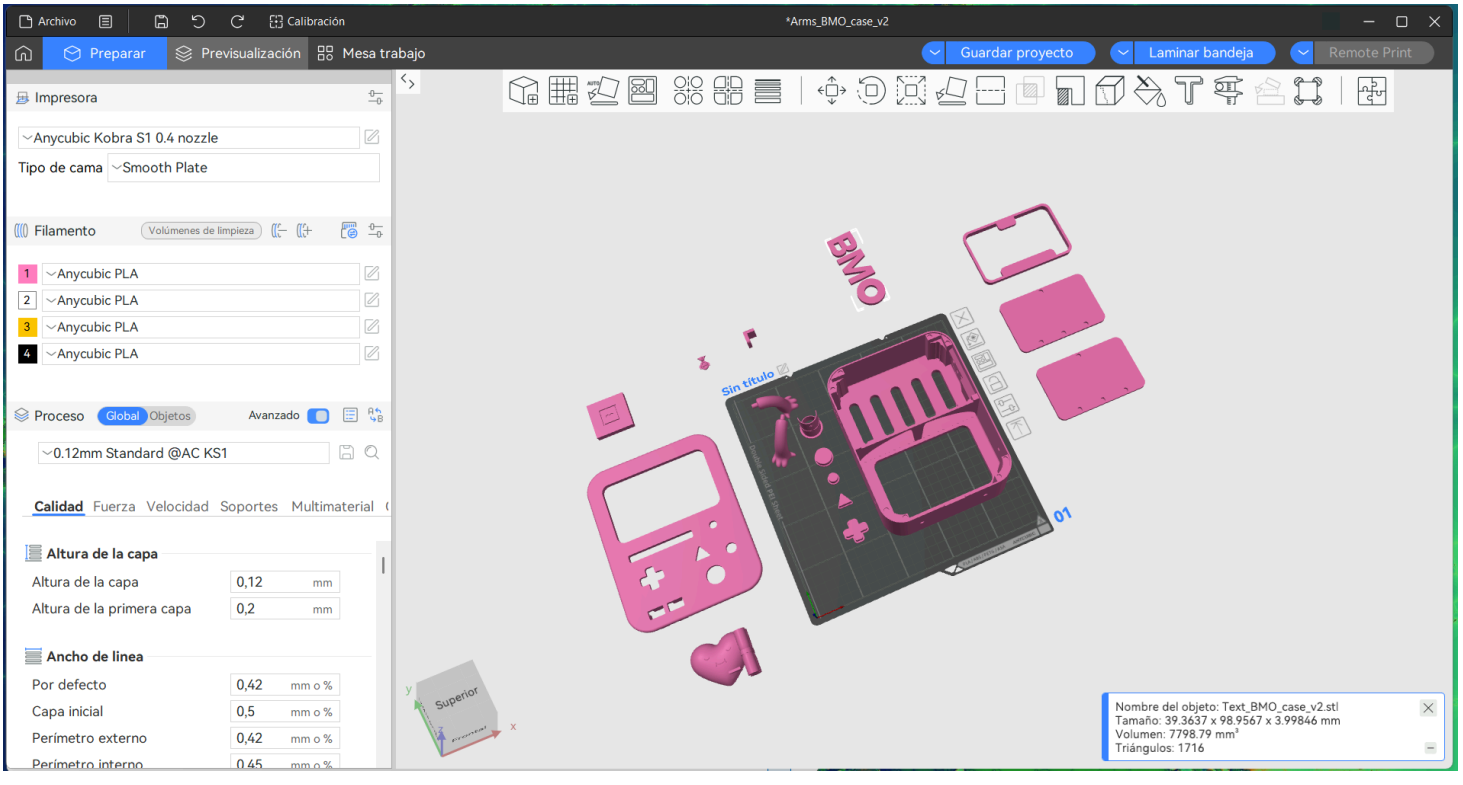
Vaig mantenir Ollama com a motor d'execució dels models perquè gestiona de manera automàtica tasques que, altrament, hauria hagut d'implementar a mà, com ara la descàrrega dels models, la seva quantització (és a dir, la reducció de la precisió numèrica dels seus paràmetres interns per fer-los més lleugers) i la càrrega i alliberament de memòria RAM quan el model no s'utilitza. Vaig conservar també els dos models que ja preveia el projecte original, gemma3:1b (815 MB) i moondream (1,7 GB): tots dos són models de mida reduïda, pensats per oferir un rendiment raonable en maquinari modest com la Raspberry Pi 5 de 8 GB de RAM que constitueix l'objectiu final del projecte, de manera que no vaig veure motiu per valorar-ne d'altres de més grans.

Pel que fa a Piper, vaig decidir mantenir la mateixa versió que ja preveia el projecte original, en lloc de canviar a una bifurcació (fork) més moderna del mateix programa. Ho vaig descartar perquè aquesta bifurcació distribueix el programa sota una llicència diferent (GPL-3.0, en lloc de la llicència MIT del projecte original), i no vaig trobar cap error ni funcionalitat concreta que justifiqués aquest canvi de llicència.

Finalment, per a la detecció de la paraula d'activació amb openWakeWord, vaig optar pel motor d'execució ONNX en lloc de l'alternatiu TFLite, atesos els problemes coneguts de compatibilitat d'aquest darrer en arquitectures ARM; tot i que en un ordinador x86_64 aquest problema concret no es donaria, vaig preferir mantenir des del principi la mateixa configuració que utilitzaria finalment a la Raspberry Pi, per no haver de gestionar dues configuracions diferents segons l'entorn.

3.1.4. Aspectes pendents del disseny

Alguns elements del disseny de l'assistent només els podré completar i verificar quan munti el sistema físicament dins la carcassa de BMO i l'executi sobre la Raspberry Pi 5 definitiva. En primer lloc, la captura d'imatges amb la càmera: el codi encara crida directament una eina de línia d'ordres específica de la Raspberry Pi (rpicam-still), inexistent en un ordinador d'escriptori convencional, de manera que aquesta funcionalitat no l'he pogut provar encara amb una càmera real. En segon lloc, la personalització de la interacció amb l'assistent: tant la paraula d'activació ("Hey BMO", en lloc de la genèrica "Hey Jarvis" que faig servir de moment) com la veu de síntesi pròpia de BMO requeririen entrenar models específics a partir de gravacions pròpies, una tasca que abordo a l'apartat 3.2.5. Finalment, preveig fixar de manera explícita, al fitxer de dependències del projecte, les versions concretes de totes les llibreries que ja he comprovat que funcionen correctament en aquesta fase, per garantir que la instal·lació final a la Raspberry Pi 5 reprodueixi exactament el mateix comportament que he aconseguit durant el desenvolupament.



3.2. Procés de construcció i posada en marxa

3.2.1. Origen del projecte i primers passos

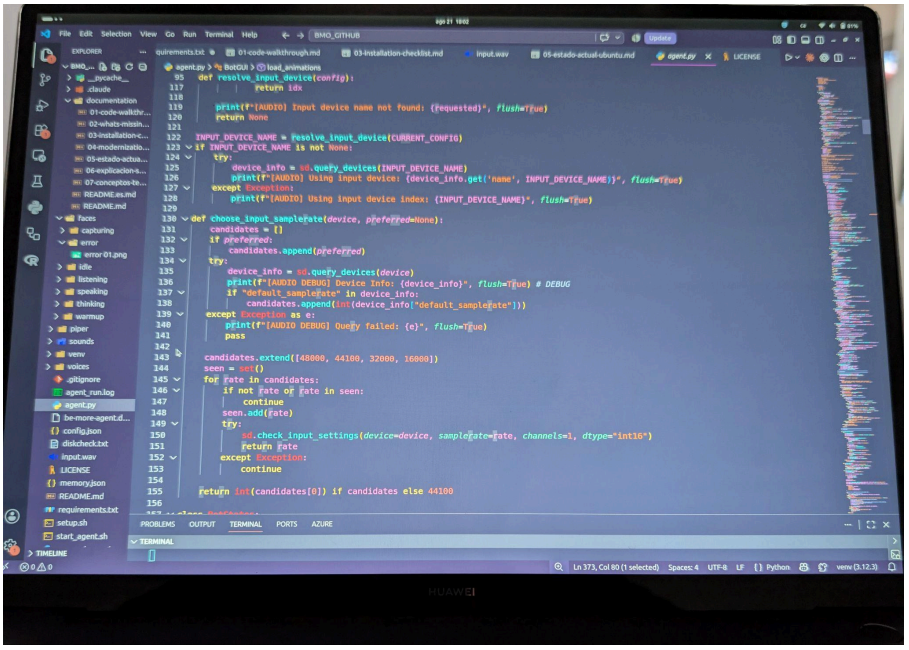
Un cop decidit el disseny i els components que formarien part de l'assistent, el primer pas pràctic va ser la impressió 3D de les peces que componen l'estructura física de BMO. Inicialment, em vaig dirigir a Can Xic, un espai que disposava d'impressores 3D, però un cop allà em van informar que les impressores s'havien traslladat a una altra ubicació. Vaig acabar trobant CienciaLab, on em van atendre molt bé i em van concretar un dia i una hora per començar la impressió. Després de dos dies de feina, vaig disposar de totes les peces impreses amb el color exacte que buscava i el material exacte que volia, aquest era PLA (àcid polilàctic) —pràcticament idèntic al del personatge original— i, en revisar-les amb detall a casa, vaig comprovar que totes estaven correctes, sense deformacions ni errors d'impressió.

Mentre s'imprimeixen les peces, vaig aprofitar per començar a treballar en paral·lel amb la part de programari, ja que ambdós processos poden avançar de manera independent.

3.2.2. Revisió i reescriptura del codi original

El punt de partida per a la part de programari va ser el codi compartit a la descripció del vídeo de referència, basat en Ollama i pensat per executar-se des de Visual Studio Code, un editor de codi molt utilitzat en programació. En revisar-lo amb detall, però, vaig detectar diversos problemes greus:

- El codi estava desordenat i mal estructurat, cosa que en dificultava molt la comprensió i la depuració.
- La veu configurada per defecte no era la de BMO, sinó una veu genèrica que no tenia res a veure amb el personatge.
- Diverses parts del codi estaven desactualitzades: feien servir eines o llibreries que ja no funcionaven correctament o que havien quedat obsoletes des que es va publicar el vídeo original.



Davant d'aquesta situació, vaig haver de reescriure bona part del codi pràcticament des de zero. Per fer-ho de manera eficient, vaig comptar amb l'ajuda de Claude, una intel·ligència artificial conversacional, que em va permetre identificar amb rapidesa quines parts del codi fallaven i per què, com també trobar alternatives actualitzades i funcionals.

Entre els canvis més importants fets durant aquesta fase hi ha:

Reconeixement de veu. El codi original esperava fer servir whisper.cpp, un programa que calia compilar manualment seguint un procés complex, i que ni tan sols estava inclòs en el projecte. En la pràctica, això feia que l'assistent no arribés mai a "sentir" res del que se li deia: la transcripció sortia sempre buida. Vaig substituir-ho per faster-whisper, una alternativa molt més senzilla d'instal·lar (n'hi ha prou amb una simple instal·lació via pip, sense necessitat de compilar cap component en C++). Utilitzo el model "base", en mode int8 (és a dir, amb els càlculs interns simplificats per guanyar velocitat, assumint una petita pèrdua de precisió), ja que m'interessa que l'assistent respongui amb rapidesa més que no pas que transcrigui de manera perfecta.

So d'error. El codi original ja preveia un so d'avís quan es produïa un error (l'assistent passava a l'estat ERROR), però aquest so mai arribava a reproduir-se perquè no estava connectat correctament a la resta del programa. Vaig corregir aquesta connexió perquè, ara, cada vegada que l'estat canvia a ERROR, es reprodueix un so aleatori d'avís.

Eina de cerca web. El codi feia servir una llibreria de cerca a internet (duckduckgo_search) que havia quedat obsoleta i que el mateix mantenidor havia canviat de nom. Vaig actualitzar la crida perquè utilitzés el nom nou de la llibreria (ddgs), mantenint exactament la mateixa funcionalitat.

Ruta de l'arxiu de veu. La configuració apuntava a un arxiu de veu (bmo-custom.onnx) que en realitat mai s'havia arribat a descarregar amb aquest nom exacte, cosa que provocava un error en intentar carregar-lo. Vaig corregir la configuració perquè apuntés a l'arxiu de veu que sí que es descarregava correctament per defecte, mentre no disposés encara de la veu definitiva de BMO.

Aquestes mancances es corresponen, en bona part, amb els deu errors metodològics i tècnics més habituals que es poden trobar en un projecte d'aquest tipus quan es parteix d'un codi obtingut de tercers sense revisar-lo a fons prèviament.

3.2.3. Instal·lació i configuració dels components de programari

Un cop identificats els problemes del codi, vaig muntar un entorn de proves en un ordinador portàtil amb Ubuntu, abans de traspassar res a la Raspberry Pi, seguint l'estratègia de desenvolupament explicada a l'apartat 3.1.1.

Abans d'instal·lar cap component d'intel·ligència artificial pròpiament dit, vaig haver d'instal·lar un seguit de paquets del sistema operatiu necessaris perquè la resta de programari es pogués compilar i funcionar correctament: eines de desenvolupament (compilador, cmake), llibreries d'àlgebra lineal, i, especialment, les llibreries de so libasound2-dev i portaudio19-dev. Aquestes dues últimes són imprescindibles perquè la llibreria Python que gestiona el micròfon (sounddevice) es pugui comunicar correctament amb el sistema de so ALSA del sistema operatiu; sense elles, la instal·lació d'aquesta llibreria seria incompleta i, tot i instal·lar-se sense errors aparents, no seria capaç d'accedir realment al micròfon.

Vaig instal·lar Ollama mitjançant l'script oficial del projecte, que s'encarrega de gestionar la descàrrega, la quantització i la càrrega en memòria dels models de llenguatge de manera automàtica. Hi vaig descarregar els dos models ja comentats a l'apartat 3.1.3: gemma3:1b, per al xat de text normal, i moondream, que s'activa quan l'assistent ha de processar una imatge.

faster-whisper el vaig instal·lar directament amb pip, sense necessitat de compilar res, tal com esperava després de la revisió del codi original (apartat 3.2.2).

Piper, el programari de síntesi de veu, es descarrega automàticament només quan el sistema detecta l'arquitectura de la Raspberry Pi (aarch64); per poder-lo provar a l'ordinador portàtil (arquitectura x86_64), el vaig haver d'instal·lar manualment amb la versió equivalent per a aquesta arquitectura, mantenint la mateixa versió que ja preveia el projecte (vegeu apartat 3.1.3 sobre la llicència).

openWakeWord, el sistema de detecció de la paraula d'activació, ja formava part del codi original. Durant aquesta fase en vaig verificar el funcionament amb un petit script de prova que escolta el micròfon durant uns segons i mostra en temps real les puntuacions de detecció, cosa que resulta molt útil per depurar aquesta part sense haver d'engegar l'aplicació sencera. De moment, el sistema continua fent servir la paraula d'activació per defecte ("Hey Jarvis"), ja que encara no he entrenat una paraula pròpia ("Hey BMO"), tasca que aborda l'apartat 3.2.5.

Finalment, vaig instal·lar totes les llibreries de Python dins d'un entorn virtual aïllat (venv), seguint el fitxer de dependències requirements.txt del projecte. Cal remarcar que aquest fitxer, tal com el vaig rebre, no fixava cap versió concreta de les llibreries, de manera que les versions que finalment vaig instal·lar no responen a una elecció deliberada per part meva, sinó simplement a les darreres versions disponibles compatibles amb la versió de Python que feia servir en el moment de la instal·lació. Per aquest motiu, tal com ja apunto a l'apartat 3.1.4, preveig fixar aquestes versions de manera explícita més endavant.

3.2.4. Resolució d'un problema de maquinari: saturació del micròfon

Durant les primeres proves completes del sistema, el flux general de funcionament arribava a executar-se d'extrem a extrem —és a dir, es detectava la paraula d'activació, s'enregistrava l'àudio, es transcrivia, es generava una resposta i aquesta es reproduïa en veu alta—, però la transcripció que obtenia sortia buidament o completament sense sentit la major part de les vegades. En comptes de donar per fet que es tractava d'un error de configuració del programa de reconeixement de veu, vaig optar per inspeccionar directament el fitxer d'àudio que el sistema enregistrava a cada torn de conversa. En fer-ho, vaig comprovar que l'àudio captat no contenia, en realitat, cap veu reconeixible, sinó un soroll constant i molt intens, semblant a l'estàtica d'una ràdio mal sintonitzada.

Investigant la causa d'aquest problema, vaig localitzar l'origen en la configuració del maquinari de so del sistema operatiu (ALSA): dos controls de guany d'entrada del micròfon —anomenats Mic Boost Volume i Capture Volume— es trobaven configurats al seu valor màxim possible, cosa que suposava un guany combinat d'aproximadament 60 decibels. Aquest nivell de guany és més que suficient per saturar el senyal captat per qualsevol micròfon integrat en un ordinador portàtil amb el simple soroll ambiental d'una habitació, produint exactament l'efecte de saturació (clipping) explicat a l'apartat 2.2.2 d'aquest treball: el senyal real "s'escapa" del rang numèric màxim que es pot emmagatzemar, i el que se sent, en lloc de la veu, és un soroll o crepitació. Vaig comprovar que es tracta, de fet, d'un problema força conegut i documentat en ordinadors portàtils que munten el xip de so Realtek ALC256, i no pas d'un error del meu codi ni de la manera com configurava la llibreria d'accés al micròfon.

La solució que vaig aplicar va consistir a reduir manualment aquests dos controls de guany mitjançant una eina de configuració d'àudio del sistema operatiu (amixer): vaig reduir el control Mic Boost Volume del seu valor màxim a zero, eliminant completament l'amplificació addicional, mentre que el control Capture Volume el vaig anar reduint progressivament, provant diversos valors fins a trobar-ne un que oferís un equilibri adequat. Concretament, vaig comprovar que un valor inferior produïa un volum d'enregistrament massa baix, mentre que un valor superior tornava a provocar una saturació lleu en les síl·labes pronunciades amb més intensitat; el valor que finalment vaig escollir va ser el que, en les proves que vaig fer, no presentava cap mostra saturada i produïa transcripcions clares i correctes. Finalment, vaig fer persistent aquest ajust mitjançant l'eina alsactl, de manera que els valors de guany correctes es mantinguessin fins i tot després de reiniciar l'ordinador.

Aquest episodi il·lustra bé un dels aspectes pràctics més rellevants de treballar amb sistemes de reconeixement de veu: sovint, els problemes que semblen de programari (una intel·ligència artificial que "no entén res") tenen, en realitat, un origen purament de maquinari, i només es poden diagnosticar correctament inspeccionant directament les dades (en aquest cas, l'àudio enregistrat) en lloc de donar per fet on es troba l'error.

3.2.5. Cerca de la veu de BMO

Un cop vaig tenir el sistema funcionant amb la veu anglesa per defecte (en_GB-semaine-medium.onnx), em vaig proposar crear una veu pròpia per a BMO, en castellà, ja que la veu original en anglès no s'adeia gaire amb la idea que tenia del personatge.

El primer intent va consistir a modificar directament el model de veu en castellà que ja incorporava Piper, amb l'ajuda de Claude Code, pel fet que el resultat inicial sonava massa robòtic i poc natural. Vaig aconseguir millorar-ne una mica l'entonació, però algunes paraules continuaven sonant-hi de manera estranya. En investigar-ne la causa, vam descobrir que el problema venia de la variant de l'idioma configurada: el model estava fixat com a ES (lat) —és a dir, castellà d'Amèrica Llatina— en lloc d'ES (ES) —castellà d'Espanya—, cosa que feia que certes paraules es pronunciessin amb una fonètica que no corresponia a la que jo buscava. Vaig corregir aquesta configuració perquè apuntés a la variant peninsular, però, tot i el canvi, la veu continuava sonant massa robòtica i no acabava de convèncer-me.

Davant d'això, vaig plantejar-me si seria possible extreure la veu directament dels episodis originals de la sèrie Hora d'Aventures, doblada en castellà, per entrenar-hi un model de veu propi. Vaig descartar aquesta opció perquè hauria suposat una feina extremadament llarga i costosa: la sèrie té més de 500 episodis, i entrenar un model de veu de qualitat requereix disposar d'una quantitat considerable d'àudio net, ben segmentat i sense sorolls de fons ni música, cosa que no és senzilla d'obtenir directament dels episodis.

Buscant alternatives, vaig trobar una pàgina web que oferia una pista d'àudio aïllada amb la veu de la dobladora original del personatge BMO en castellà, sense diàleg d'altres personatges ni música de fons, que hauria estat un material ideal per entrenar-hi una veu. Tanmateix, aquest material estava protegit per drets d'autor, de manera que no el vaig poder utilitzar per a aquest projecte.

Finalment, davant la impossibilitat d'obtenir un material d'àudio adequat i lliure de drets per entrenar una veu pròpia en castellà, vaig optar per mantenir la mateixa veu que ja incorporava el projecte original de referència, en anglès. Aquest procés em va fer adonar d'una limitació pràctica important a l'hora de personalitzar un assistent d'IA local: encara que el maquinari i el programari necessaris siguin accessibles, la creació de continguts personalitzats (com una veu concreta) topa sovint amb restriccions de propietat intel·lectual que no es poden solucionar únicament amb coneixements tècnics.

3.2.6. Trasllat a la Raspberry Pi

Un cop tot el sistema funcionava correctament en l'entorn de proves d'Ubuntu, tal com preveia l'estratègia de desenvolupament explicada a l'apartat 3.1.1, va arribar el moment de traspassar-ho tot a la Raspberry Pi 5, el dispositiu final on ha de viure BMO. Aquest pas, però, no va ser tan directe com s'esperava, i hi van sorgir diversos problemes nous:

- La càmera no funcionava correctament.
- El micròfon no responia.
- En alguns casos, directament no s'aconseguia connectar al codi de la Raspberry Pi.

En un primer moment, aquests símptomes semblaven indicar problemes independents entre si —un de maquinari (càmera), un altre de configuració d'àudio (micròfon) i un altre de connectivitat—, cosa que en dificultava el diagnòstic. Després d'investigar-ne l'origen, però, vam descobrir que dos d'aquests símptomes tenien, en realitat, una única causa comuna, molt més senzilla del que semblava: en crear la imatge del sistema operatiu per a la Raspberry Pi, el meu pare hi havia establert una contrasenya que després no recordava, cosa que impedia l'accés correcte al dispositiu i, per extensió, la impossibilitat de connectar-s'hi.

Un cop recuperada i canviada la contrasenya, vam poder accedir correctament a la Raspberry Pi i continuar amb la configuració dels components pendents. El problema del micròfon, en canvi, no tenia res a veure amb la contrasenya: un cop vaig poder accedir de nou al sistema, vaig comprovar que el codi no tenia ben configurat el dispositiu de micròfon per al maquinari de la Raspberry Pi (a diferència del portàtil, on la configuració ja s'havia ajustat durant les proves inicials, vegeu apartat 3.2.4), i el vaig haver de corregir directament al codi. La càmera, per la seva banda, va continuar donant problemes fins i tot després de resoldre la contrasenya i el micròfon, de manera que vam optar per comprar-ne una de nova. En el moment d'escriure aquest apartat, encara no s'ha pogut acabar de comprovar si aquesta nova càmera funciona correctament amb el sistema, tot i que l'expectativa és que sí.

Aquest episodi, tot i semblar anecdòtic, il·lustra un aprenentatge pràctic força habitual en projectes d'aquest tipus: símptomes que en un primer moment semblen relacionats (tres coses que fallen alhora en traslladar el sistema a un nou dispositiu) poden tenir, en realitat, causes completament independents entre si —una contrasenya oblidada, una configuració de codi pensada només per a un altre maquinari, i un component físic potencialment defectuós—, i cal investigar-les per separat en lloc de donar per fet que comparteixen un únic origen.

3.3. Anàlisi i resultats de l'assistent

3.3.1. Metodologia de les proves

Un cop resolts els problemes de connexió i de micròfon descrits a l'apartat 3.2.6, i amb l'única excepció encara pendent de la càmera, vaig poder començar a fer les primeres proves reals de conversa amb l'assistent ja muntat sobre la Raspberry Pi. Aquestes proves van consistir, senzillament, a parlar-hi de manera natural i observar tant la resposta que donava com el comportament de la carota animada a la pantalla, per comprovar si es corresponia amb l'estat en què s'hauria de trobar el sistema en cada moment (vegeu la màquina d'estats descrita a l'apartat 2.6.1 i 3.1.2).

3.3.2. Primeres proves de conversa

La primera prova va consistir a saludar l'assistent amb un "hello, how are you?". Vaig comprovar que, per tal que l'assistent entengués la pregunta, calia formular-la en anglès; si es feia en català o en castellà, el sistema no l'entenia. Això passa perquè el model de reconeixement de veu utilitzat (faster-whisper, en la seva variant "base.en", vegeu apartat 3.1.3) està entrenat i optimitzat específicament per a l'anglès, i no per a altres idiomes. L'assistent va respondre correctament a la pregunta, i, de fet, ho va fer amb més rapidesa de la que m'esperava.

3.3.3. Detecció d'un error

Després d'unes quantes proves més, el meu pare li va preguntar "com estàs?" en català. En aquest cas, l'assistent no va respondre bé (tal com era d'esperar, ja que no s'havia formulat en anglès), però, a més, la carota de la pantalla va canviar durant una fracció de segon a l'expressió que l'assistent fa servir quan es disposa a fer una fotografia —corresponent a l'estat CAPTURING de la màquina d'estats—, per després tornar de seguida a la seva expressió normal.

Aquest comportament em va cridar l'atenció, ja que en cap moment s'havia demanat a l'assistent que fes cap fotografia. Vaig decidir investigar-ne l'origen amb calma, revisant el codi que gestiona els canvis d'expressió facial. Després d'una estona d'anàlisi, vaig descobrir que el programa intentava mostrar dues expressions diferents gairebé alhora, i que, en triar-ne una a l'atzar per mostrar-la primer, en aquell cas concret havia sortit la cara de fer una fotografia; com que la càmera no estava detectada (l'única part del sistema encara pendent de resoldre), aquesta expressió desapareixia gairebé immediatament, cosa que n'explicava la brevetat.

Per solucionar aquest problema, vaig modificar el codi perquè la cara de fer una fotografia només es pogués activar quan el contingut de la conversa fes referència explícita a fotografies o a la càmera, i mai en cap altra situació. En repetir la prova, l'assistent ja no mostrava aquesta expressió de manera inesperada en preguntes que no hi tenien res a veure.

Ara bé, aquest canvi va introduir un segon problema: com que havia restringit l'aparició d'aquesta expressió a un únic cas molt concret, no havia previst cap mecanisme perquè tornés a desaparèixer si la càmera no responia. Com a conseqüència, un cop l'assistent mostrava la cara de fer una fotografia, s'hi quedava per sempre, ja que mai es donava la condició (detectar la càmera) que l'hauria de fer tornar a l'estat normal. Per resoldre-ho definitivament, vaig afegir una condició addicional perquè, si al cap d'uns tres segons el sistema no detectava cap càmera disponible, l'expressió facial tornés automàticament al seu estat per defecte.

Aquest cas em sembla especialment il·lustratiu del tipus de dificultats que sorgeixen en un sistema basat en una màquina d'estats amb components que s'executen en paral·lel (vegeu apartat 3.1.2): un error que, superficialment, semblava un comportament estrany i puntual (una cara "equivocada" durant una fracció de segon) va resultar tenir una causa ben concreta —una condició de carrera entre dues expressions que competien per mostrar-se alhora—, i la seva solució, alhora, va generar un segon problema que no havia previst inicialment. Aquesta experiència reflecteix bé com, en la pràctica, resoldre un error sovint no és un procés d'un sol pas, sinó una successió de petits ajustos, cadascun dels quals cal tornar a provar abans de donar el problema per tancat.

4. CONCLUSIONS

Un cop finalitzat el treball, puc dir que he assolit la major part dels objectius que em vaig plantejar a la introducció, tot i que amb algun matís que val la pena comentar.

Pel que fa al primer objectiu, conèixer els fonaments de funcionament dels models de llenguatge (LLM) i dels sistemes de reconeixement i síntesi de veu, considero que l'he assolit plenament: al llarg de la part teòrica he pogut explicar amb detall com funciona un LLM, com es transcriu la veu a text i com es genera la parla a partir d'un text, i aquest coneixement m'ha estat imprescindible per entendre i corregir el codi de l'assistent durant la part pràctica.

Quant al segon objectiu, analitzar les diferències tècniques entre la intel·ligència artificial local i la basada en el núvol, també el considero assolit des del punt de vista teòric (apartat 2.3), ja que he pogut identificar i explicar les diferències clau en termes de privadesa, cost, rendiment i accessibilitat. Ara bé, cal assenyalar que la prova comparativa pràctica que tenia previst fer entre el meu assistent local i un assistent comercial del núvol, finalment no l'he pogut dur a terme per manca de temps. Ho comento amb més detall més endavant, a l'apartat d'incidències.

El tercer objectiu, dissenyar i construir un prototip funcional d'assistent d'IA amb Raspberry Pi, l'he assolit satisfactòriament: l'assistent funciona, escolta, entén, respon i parla, tal com es descriu a la part pràctica d'aquest treball.

El quart objectiu, avaluar el rendiment i les limitacions del prototip mitjançant proves pràctiques, l'he assolit parcialment: he pogut fer diverses proves de conversa real amb l'assistent i detectar-hi i corregir-hi errors concrets (com el de la cara de fer una fotografia, explicat a l'apartat 3.3.3), però encara em queda pendent de comprovar el funcionament de la càmera amb el nou component que vaig haver de comprar.

Finalment, el cinquè objectiu, reflexionar sobre la dependència actual de la societat envers la intel·ligència artificial, el desenvolupo amb més profunditat en la reflexió personal d'aquest mateix apartat.

Valoració del procés

Si hagués de valorar el procés en conjunt, diria que es tracta d'un projecte que no es pot fer en pocs mesos, sinó que cal anar-lo aprenent a poc a poc, pas a pas. Quan vaig començar, no en sabia gairebé res: no sabia com connectar els cables, no sabia programar el codi que calia, ni tan sols sabia com generar una veu per a l'assistent. Amb temps i esforç, però, es pot arribar a entendre-ho tot, encara que es comenci literalment de zero. Un dels aprenentatges més importants que m'emporto d'aquest treball és que s'aprèn moltíssim dels mateixos errors: cada vegada que alguna cosa fallava (el micròfon, el codi, la contrasenya, la cara de la càmera...), investigar-ne la causa em feia entendre molt millor com funcionava realment el sistema, molt més que si tot hagués funcionat a la primera. Al final, s'aconsegueixen coses que un mateix mai s'hauria imaginat que podria arribar a fer. Ara que ja tinc l'assistent construït i funcionant per a mi mateix, tinc pensat continuar-hi fent millores de tota mena en el futur.

Incidències

Durant el desenvolupament del treball van sorgir diverses incidències, la majoria de les quals ja s'han descrit amb detall a la part pràctica: la manca d'una veu de BMO en castellà, per la impossibilitat d'obtenir material d'àudio lliure de drets per entrenar-la (apartat 3.2.5); els problemes de connexió, micròfon i càmera en traslladar el sistema a la Raspberry Pi, amb causes independents entre si (apartat 3.2.6); i la impossibilitat de dur a terme, per manca de temps, la prova comparativa pràctica entre l'assistent local i un assistent comercial del núvol, que hauria estat un complement interessant a l'estudi comparatiu teòric de l'apartat 2.3.

Prospectiva

De cara al futur, m'agradaria continuar millorant l'assistent en diversos aspectes: acabar de verificar i, si cal, resoldre el funcionament de la nova càmera; entrenar una paraula d'activació pròpia ("Hey BMO") en lloc de la genèrica que faig servir de moment; i, si trobo la manera d'obtenir-ne el material necessari sense infringir drets d'autor, continuar intentant aconseguir una veu pròpia de BMO en català o en castellà. També crec que seria interessant, més endavant, dur a terme la prova comparativa amb un assistent comercial que no vaig poder completar en aquest treball.

Reflexió personal

Aquests darrers anys, la intel·ligència artificial ha generat molt de debat, especialment pel que fa al seu impacte sobre els llocs de treball. És cert que, avui dia, ja hi ha establiments on els cambrers han estat substituïts per robots amb intel·ligència artificial, i aquest exemple posa de manifest que la preocupació per la pèrdua de llocs de treball no és exagerada. Amb tot, després de fer aquest treball, la meva opinió és que la intel·ligència artificial no és, en si mateixa, ni bona ni dolenta: es tracta d'un suport que ens pot ajudar en moltes tasques, alhora que ens en pot treure d'altres. No crec, però, que en el futur tot acabi essent fet per una intel·ligència artificial, llevat que s'arribi a descobrir alguna manera de fer-la molt més fiable i intel·ligent del que és ara; mentre això no passi, dubto que la IA arribi a perjudicar-nos de manera significativa. En qualsevol cas, construir el meu propi assistent d'IA local m'ha ajudat a desmitificar-la: darrere de la "màgia" que sembla que tenen aquests sistemes, hi ha decisions tècniques molt concretes, models petits amb limitacions clares, i molts errors que cal anar corregint un per un, exactament igual que en qualsevol altre projecte tecnològic.